National University of Computer and Emerging Sciences

Assignment No. 4

Generative AI

AI4009

Spring 2026

Submission Instructions:

- Submit the complete .ipynb file, which should be named AI_ASS01_XXF_YYYY, where XX represents the batch and YYYY represents the roll number.
- Submit a Word file containing the GitHub repository link, as well as the Medium and LinkedIn post links
- This is a group assignment (max 2 members allowed).
- Any submission not following the submission instructions will not be evaluated.
- Explore the Datasets completely before starting to code.
- You can explore more datasets if needed and must explain your reasoning of using them but there is a constraint attached datasets must need to be used.
- **Submit your own work. Accept your current limitations. Grow from your mistakes.**
- **Plagiarism is strictly prohibited and lead to Straight Zero (^_^).**

Question: Diffusion Models for High-Resolution Image Generation & Reconstruction

Objective: The objective of this assignment is to design and implement a diffusion-based generative model that learns to generate and reconstruct high-resolution images by progressively removing noise. You will:

- Understand the forward (noising) and reverse (denoising) processes
- Implement a Denoising Diffusion Probabilistic Model (DDPM) using PyTorch
- Train the model on a real-world dataset
- Generate high-quality images from pure noise

The system must be implemented using **base PyTorch layers only** (no direct use of pretrained diffusion pipelines).

1. Environment Setup

- **Platform:** https://www.kaggle.com/
- **Accelerator:** GPU T4 x2 (Dual GPU)
- **Dataset:** Use any of the following Dataset
  - CelebA-HQ: https://www.kaggle.com/datasets/denislukovnikov/celebahq256-images-only
  - FFHQ (High-quality faces): https://www.kaggle.com/datasets/greatgamedota/ffhq-face-data-set
  - WikiArt Dataset: https://www.kaggle.com/datasets/sairam3/wikiart

2. Model Architecture

➢ Diffusion Model (DDPM)

The model consists of the two main processes:

Forward Process (Noising):

- Gradually adds noise to the image over multiple steps
- Implemented as a fixed process

Reverse Process (Denoising):

- Learns to remove noise step-by-step
- Implemented using a neural network

➢ Network Backbone (U-Net)

Students must implement simplified U-Net architecture.

Configuration:

- Input: Noisy image + timestep information
- Output: Predicted noise
- Channel progression: 64 → 128 → 256
- Use residual blocks
- Include time-step embeddings
- Downsampling and upsampling layers must be implemented

3. Implementation Tasks

Part 1: Data Preprocessing

1. Resize images to:
   - 128 × 128 (minimum) OR
   - 256 × 256 (recommended)
2. Normalize pixel values
3. Create dataloader with batching

Part 2: Forward Diffusion

1. Define a noise schedule
2. Add noise progressively to images
3. Generate noisy versions of input images at different timesteps
4. Visualize at least 5 noising steps

Part 3: Reverse Process

1. Implement U-Net
2. Input:
   - Noisy Image
   - Timestep
3. Output:
   - ➢ Predicted Noise
4. Train model to learn noise removal

Part 4: Training Setup

Loss Function

- Mean Squared Error (MSE)
- Compare predicted noise with actual noise

Optimizer

- Adam or AdamW

Scheduler

- Optional (Cosine or StepLR)

Required Training Techniques (to fit Kaggle T4×2)

- Mixed Precision Training
- Small batch size (16–32)
- Optional gradient clipping
- Reasonable number of timesteps (200–500 recommended)

Part 5: Image Reconstruction Task (CORE TASK)

You will be provided with a target image. Your Task is to:

1. Start from random noise
2. Apply reverse diffusion process
3. Generate an image that closely resembles the target

Part 6: Image Generation Task

1. Generate at least 5 new images from pure noise
2. Ensure outputs are:

- Sharp
- Visually meaningful
- High quality

4. Visualization Module

Create a utility that shows:

- Original image
- Noisy versions at different steps
- Intermediate denoising steps
- Final generated output

Display at least:

- 5 forward diffusion steps
- 5 reverse diffusion steps
- 5 generated images

Deliverables

1. **Jupyter Notebook:**

- Complete PyTorch implementation.
- Clean, well-structured code

2. **Training Logs:**

- Loss vs epochs plot

- Evidence of stable training

3. **Results:**
   - At least 5 generated images
   - Reconstruction of target image
   - Side-by-side comparison:
     - Target vs Generated
4. **Quantitative Evaluation:**

   Compute:
   - PSNR Score
   - SSIM Score
5. **App Deployment:**
   - Build a Gradio or Streamlit App that:
     - Starts from random noise
     - Generates an image
     - Displays intermediate denoising steps

Important Instructions

- Do NOT use pretrained diffusion models
- Do NOT use libraries like HuggingFace Diffusers directly
- You may use PyTorch utilities only
- Code must be your own implementation

Bonus (Optional)

- Implement improved sampling (faster generation)
- Try different noise schedules
- Train on higher resolution (256×256)
- Add conditioning (class or style-based generation)